

## **1. Project Architecture**

Our project has three separate MiniScheme interpreters, each built in a different programming paradigm. The C++ version is stored in the procedural folder, the Java version is stored in oop, and the Common Lisp version is stored in functional. They all use the same public test files and get called through the shared run\_all.sh script. The runner supports list-cases, run-case, and compare-case, which allows us to test one implementation or compare all three through a single command-line interface.

## **2. Implementation Overview by Paradigm**

Procedural (C++) Written by: Emilio Calderon

The C++ implementation uses a procedural style, where the program follows clear steps for reading, parsing, and evaluating MiniScheme expressions. It reads the input file, splits the input into tokens, parses expressions into a Node-style tree structure, and evaluates each node recursively. It also uses an environment structure to store variables and functions while handling required features like if, let, lambda, recursion, and cond.

Object-Oriented (Java) Written by: Stephanie Esquivel

The Java implementation follows a three-stage pipeline; tokenizer first converts raw .scm file text into a flat list of tokens, stripping comments and separating parentheses. The parser then reads the tokens one at a time and builds an AST made of Expressions objects. The evaluator distributed across the expressions class uses polymorphism. Then each subclass overrides evaluate(env) with its own logic. The main loop calls

express.evalaute(globalEnv) and java returns the correct version. Key OOP decisions include encapsulating the Environment's HashMap behind define() and lookup() to protect the scope chain from direct access and using inheritance through MException extending RuntimeException to signal MiniScheme-specific errors cleanly. Lexical scope is implemented through closures that save the environment at lambda creation time, and recursion is supported because define adds functions to the global environment before they are called.

Functional (Common Lisp) Written by: Chazz Carrizales

The Common Lisp implementation uses recursive evaluation to process MiniScheme expressions. Expressions are read from the input file, evaluated, and returned with a result with a type or an error. The evaluator uses an environment to keep track of bindings for variables and functions, and lambdas have closures, so they keep the environment where they were created. This supports lexical scope, let, functions, define, recursion, and the cond add-on. The implementation also supports results and errors through run\_all.sh.

### **3. Comparative Analysis**

The three implementations showed how MiniScheme language rules can be represented differently depending on the programming paradigm. The C++ version was more step-by-step, with explicit parsing, control flow, and managing how expressions are evaluated. The Java version organized the interpreter around classes like expressions and parsing, which made the responsibilities clearer. The Common Lisp version was more natural, since Scheme expressions are list-based, though Lisp syntax and the functional programming

style are harder to work with.

#### **4. Add-On Integration**

The add-on for our project was `cond`. We handled `cond` by reading each condition-result in order, checking the condition, and returning the result for the first true condition. In the C++ version, this was handled with the procedural parser and evaluator logic. In the Java version, `cond` was its own expression class, with arrays for conditions and results, plus an else result. In the Common Lisp version, `cond` has been handled recursively, using the environment to evaluate each condition and result. We tested this with the add-on cases to make sure everything worked correctly.

#### **5. Known Limitations**

The project passed the public tests for expressions, booleans, `let`, `lambda`, recursion, errors, `cond`, and challenge cases. The interpreters only support the MiniScheme subset, not the full Scheme language.

The limitations in java implementation currently support integer arithmetic only with no floating-point support. Lambda expressions support single-expression bodies only. The `cond` expression with no matching clause and no else result in parse error. Function application is limited to closures and primitive operators.

#### **6. AI / External Assistance Disclosure**

AI tools were mostly used to ask questions when stuck, or there was an explanation needed. Walking through bugs, practice presentation questions, and syntax clarification were some tasks that AI helped with.